CS/ORIE 5135 - Project Writeup

Exam Scheduling via Integer Linear Programming

Spring 2026
Alam, Sulaiman - sa2459

May 5th 2026

Github URL: `https://github.com/Sulaiman-Alam/CS-5135-Final-Project`

Problem Statement

A university must schedule m exams into n available time slots. Each exam i requires $r_i \in \mathbb{Z}_+$ proctors, and each time slot may deploy at most R proctors in total. Two exams that share at least one enrolled student cannot be assigned to the same time slot. This conflict structure is encoded in a graph $G = (V, E)$, where $V = \{1, \ldots, m\}$ and $(i, i') \in E$ if exams i and i' share a student. The objective is to assign every exam to exactly one time slot while minimizing the total number of time slots used.

1 Question 1: Natural and Clique-Strengthened Formulations

1(a) Natural ILP Formulation (F1a)

Decision variables:

$$\begin{aligned} x_{ij} &\in \{0,1\} \quad \text{exam } i \text{ is assigned to slot } j, \quad i = 1, \ldots, m,\ j = 1, \ldots, n \\ y_j &\in \{0,1\} \quad \text{slot } j \text{ is used}, \quad j = 1, \ldots, n \end{aligned}$$

Formulation:

$$\min \quad \sum_{j=1}^{n} y_j \tag{1}$$

$$\text{s.t.} \quad \sum_{j=1}^{n} x_{ij} = 1 \quad \forall\, i = 1, \ldots, m \tag{2}$$

$$\sum_{i=1}^{m} r_i\, x_{ij} \le R\, y_j \quad \forall\, j = 1, \ldots, n \tag{3}$$

$$x_{ij} + x_{i'j} \le 1 \quad \forall\, (i, i') \in E,\ \forall\, j = 1, \ldots, n \tag{4}$$

$$x_{ij},\ y_j \in \{0,1\} \quad \forall\, i, j$$

Constraint (2) ensures every exam is assigned to exactly one slot. Constraint (3) enforces the proctor budget and links x to y: if any exam is placed in slot j, then $y_j = 1$ and the total proctor load cannot exceed R. Constraint (4) prevents conflicting exams from sharing a slot.

1(b) Clique-Strengthened Formulation (F1b)

The conflict constraints (4) in F1a are pairwise: one constraint per edge $(i, i') \in E$ per slot j. These can be strengthened using *clique inequalities*.

A **clique** $C \subseteq V$ is a set of exams that are mutually conflicting, i.e., every pair $(i, i') \in C$ satisfies $(i, i') \in E$. Since no two exams in a clique can be assigned to the same slot, at most one exam from C can appear in any given slot:

$$\sum_{i \in C} x_{ij} \leq 1 \quad \forall\, j = 1, \ldots, n,\ \forall \text{ clique } C \subseteq V.$$

For a clique of size k, this single constraint dominates all $\binom{k}{2}$ pairwise edge constraints it subsumes. In practice, we restrict to **maximal cliques** (cliques not contained in any larger clique), which are found using the Bron–Kerbosch algorithm via `networkx.find_cliques`. Every edge constraint is implied by at least one maximal clique constraint, so replacing (4) with maximal clique inequalities yields a valid and stronger formulation.

Formulation F1b is identical to F1a except constraint (4) is replaced by:

$$\sum_{i \in C} x_{ij} \leq 1 \quad \forall\, j = 1, \ldots, n,\ \forall \text{ maximal clique } C. \tag{5}$$

2 Question 2: Extended Formulation

2(a) Extended Formulation (F2)

A **feasible grouping** is a subset $S \subseteq \{1, \ldots, m\}$ satisfying:

1. $\sum_{i \in S} r_i \leq R$ (proctor budget), and
2. S is a stable set in G (no two exams in S conflict).

Let $\mathcal{S}$ denote the set of all feasible groupings. Each grouping represents a valid assignment of exams to a single time slot. We enumerate $\mathcal{S}$ via recursive backtracking with pruning: starting from an empty set, we iteratively add exams that do not conflict with the current group and do not exceed the proctor budget, only considering exams with higher indices than the last added (to avoid duplicates).

Decision variable:

$$\lambda_S \in \{0, 1\} \quad S \in \mathcal{S} : \text{ grouping } S \text{ is selected as a time slot.}$$

Formulation F2 (set partitioning):

$$\min \quad \sum_{S \in \mathcal{S}} \lambda_S \tag{6}$$

$$\text{s.t.} \quad \sum_{S \in \mathcal{S}:\, i \in S} \lambda_S = 1 \quad \forall\, i = 1, \ldots, m \tag{7}$$

$$\lambda_S \in \{0, 1\} \quad \forall\, S \in \mathcal{S}$$

Constraint (7) ensures each exam appears in exactly one selected grouping.

2(b) Set Covering vs. Set Partitioning

The covering constraint (7) can be written as either an equality ($= 1$, set partitioning) or an inequality (≥ 1, set covering):

Set partitioning ($= 1$): Each exam must appear in *exactly* one selected grouping. This is stricter and more faithful to the scheduling problem, since assigning an exam to multiple slots is wasteful and infeasible.

Set covering (≥ 1): Each exam must appear in *at least* one selected grouping. This relaxes the constraint and allows an exam to be covered multiple times.

In terms of integer solutions, both formulations yield the same optimal value: any optimal set covering solution can be converted to a set partitioning solution without increasing cost, since double-covering an exam wastes a slot. However, the LP relaxations differ. The set covering LP relaxation is weaker (feasible region is larger), potentially yielding a lower LP bound. The set partitioning LP relaxation is tighter, which can reduce the number of branch-and-bound nodes needed. In our implementation we use set partitioning, as it provides a tighter relaxation and is the more natural model for this problem.

3 Question 3: Computational Results

3(a) Dataset 1 (Time Limit: 120 seconds)

Table 1 reports the LP relaxation value, branch-and-bound node count, solve time, and optimality status for all three formulations on each instance. Instances 01–03 are small (35–40 exams), 04–07 are medium (60–68 exams), and 08–10 are large (75 exams).

(i) LP Relaxation Comparison

Across all ten instances, F1a and F1b yield *identical* LP relaxation values. This indicates that, on these particular instances, the maximal clique inequalities of F1b do not tighten the LP bound beyond what the pairwise edge constraints of F1a already provide. This can occur when the conflict graph has a structure (e.g., many small or triangle-free cliques) such that maximal cliques reduce to edges, making the two formulations equivalent at the LP level.

F2 provides a strictly tighter LP relaxation on instances 01–03 (e.g., 7.18 vs. 6.32 on instance_01, and 8.12 vs. 7.88 on instance_03), while matching F1a and F1b on instances 04–10. The tighter bound on smaller instances reflects that the extended formulation more precisely captures the combinatorial structure of feasible groupings. On larger instances, the LP bounds converge, suggesting that the proctor budget constraint becomes the binding factor rather than the conflict structure.

(ii) Branch-and-Bound Node Counts and Solve Times

Despite having identical LP relaxation values, F1a and F1b exhibit very different branch-and-bound behavior. F1a solves 5 out of 10 instances to optimality within 120 seconds, while F1b solves only 1 out of 10. Notably, F1b hits the time limit on nearly every instance, often with only 1 node explored. This suggests that F1b's clique constraints, while not improving the LP bound,

Table 1: Dataset 1 Results (120s time limit). † = time limit reached.

Instance	Form.	LP Relax	Obj	Nodes	Time (s)	Optimal
instance_01	F1a	6.32	8.0	1	16.09	✓
	F1b	6.32	8.0	70	120.11	†
	F2	7.18	8.0	1	0.16	✓
instance_02	F1a	7.32	9.0	4663	120.06	†
	F1b	7.32	9.0	889	120.11	†
	F2	7.92	9.0	1	0.33	✓
instance_03	F1a	7.88	9.0	221	46.62	✓
	F1b	7.88	9.0	334	120.21	†
	F2	8.12	9.0	1	0.61	✓
instance_04	F1a	12.84	14.0	1436	120.18	†
	F1b	12.84	14.0	1	120.28	†
	F2	12.84	13.0	1	1.79	✓
instance_05	F1a	12.40	13.0	1	53.04	✓
	F1b	12.40	14.0	1	128.92	†
	F2	12.40	13.0	1	1.55	✓
instance_06	F1a	12.88	14.0	381	120.15	†
	F1b	12.88	15.0	1	120.33	†
	F2	12.88	13.0	1	2.45	✓
instance_07	F1a	12.68	14.0	1024	120.16	†
	F1b	12.68	15.0	1	120.18	†
	F2	12.68	13.0	1	5.70	✓
instance_08	F1a	15.36	16.0	1	5.14	✓
	F1b	15.36	17.0	1	120.21	†
	F2	15.36	16.0	1	7.15	✓
instance_09	F1a	14.16	15.0	1	16.33	✓
	F1b	14.16	16.0	1	120.25	†
	F2	14.16	15.0	1	2.24	✓
instance_10	F1a	14.68	16.0	564	120.17	†
	F1b	14.68	17.0	1	120.20	†
	F2	14.68	15.0	1	2.13	✓

significantly increase the per-node solve time due to the larger number of constraints, making each LP solve more expensive without a compensating reduction in nodes.

This result illustrates an important point: *equal LP relaxation values do not guarantee equal branch-and-bound performance*. The size and structure of the constraint matrix, the time required to solve each node's LP relaxation, and Gurobi's internal heuristics all contribute to overall solver performance. A formulation with more constraints can be slower per node even if its LP bound is no tighter.

F2 dominates both F1a and F1b decisively. It solves all 10 instances to optimality, always in under 8 seconds, and always with exactly 1 branch-and-bound node. This means Gurobi resolves the integer program at the LP root node without any branching, and the LP relaxation of F2 is so tight relative to the integer optimum that no branching is needed.

(iii) Scaling with Instance Size

For F1a, computational effort is irregular across instance sizes. Some large instances (08, 09) solve quickly while some medium instances (02, 04, 06, 07) hit the time limit. This unpredictability is characteristic of ILP solvers, where problem structure matters more than raw size.

F1b consistently hits the time limit regardless of size, suggesting it does not scale well even for small instances in this setting.

F2 scales well: solve times grow from 0.16s (instance_01, 2,573 groupings) to 7.15s (instance_08, 34,906 groupings) as the number of feasible groupings increases. Since F2 always terminates at 1 node, the growth in solve time is driven entirely by the increasing number of variables (groupings), not by branching complexity.

3(b) Dataset 2 (Time Limit: 600 seconds)

Table 2 reports results for F1a and F1b on the nine larger instances (80–90 exams). F2 is not applicable at these sizes because the number of feasible groupings grows exponentially with m: enumerating and storing all groupings becomes computationally infeasible both in time and memory, making the extended formulation impractical.

Table 2: Dataset 2 Results (600s time limit). † = time limit reached.

Instance	Form.	LP Relax	Obj	Nodes	Time (s)	Optimal
instance_01	F1a	16.32	17.0	1	11.05	✓
	F1b	16.32	18.0	1	600.27	†
instance_02	F1a	16.08	17.0	1	108.79	✓
	F1b	16.08	18.0	1	600.29	†
instance_03	F1a	16.28	17.0	1	95.37	✓
	F1b	16.28	18.0	1	600.29	†
instance_04	F1a	16.76	18.0	1924	600.35	†
	F1b	16.76	19.0	1	600.47	†
instance_05	F1a	16.60	18.0	3824	600.41	†
	F1b	16.60	19.0	1	600.60	†
instance_06	F1a	17.76	18.0	1	120.93	✓
	F1b	17.76	20.0	1	600.40	†
instance_07	F1a	16.88	18.0	3246	600.36	†
	F1b	16.88	18.0	1	600.40	†
instance_08	F1a	16.84	18.0	6755	600.28	†
	F1b	16.84	19.0	1	951.72	†
instance_09	F1a	18.00	19.0	986	778.86	†
	F1b	18.00	20.0	1	600.39	†

Again, F1a and F1b produce identical LP relaxation values across all instances. F1a solves 4 out of 9 instances to optimality, while F1b solves none. F1b consistently hits the 600-second time limit on every instance, and in two cases (instances 08 and 09) even exceeds the limit, which is likely due to overhead from the large number of clique constraints. This confirms the pattern observed in Dataset 1: the clique constraints of F1b significantly slow down per-node LP solves without

improving the LP bound, making F1b strictly worse than F1a in practice on these instances.

As instance sizes grow from 80 to 90 exams, F1a's node counts increase substantially (up to 6,755 nodes on instance_08), indicating that the branch-and-bound tree grows rapidly. F1b, by contrast, consistently explores only 1 node but fails to find a provably optimal solution before the time limit, suggesting that the root node LP solve itself becomes extremely slow at these sizes.

The extended formulation F2 is not applicable to Dataset 2 because enumerating all feasible groupings is exponential in m. For instances with 80–90 exams, the number of stable sets in the conflict graph that satisfy the proctor budget constraint can easily reach tens of millions or more, making enumeration, storage, and model construction computationally infeasible within any reasonable time or memory budget.

Note on AI Usage

Claude (Anthropic) was used as a supporting tool during this project, mainly to help speed up development and writing. In particular:

- **Code organization:** It was used to brainstorm and refine the overall project structure (e.g., separating formulations into `formulation_f1a.py`, `formulation_f1b.py`, `formulation_f2.py`, along with shared utilities and an experiment runner). The final structure reflects my own design choices after iteration.
- **Writeup support:** It helped with LaTeX formatting and improving the flow of some explanations, but all formulations and discussions were written and edited by me.
- **Debugging:** It was occasionally used to help diagnose issues (e.g., LP relaxation extraction and edge cases when the solver hit time limits), though fixes were implemented and verified independently.

All modeling decisions, implementations, and result interpretations were done independently. AI was used as a lightweight assistant for iteration and clarity, not as a substitute for understanding the material.